

LLM, RAG and Agentic AI Terminology

One-liner definitions with examples, tools, extra notes and architecture fit - 154 topics

This compact PDF is designed as a trainer-friendly quick reference. Each topic includes a one-line definition, a practical example, tools or models commonly associated with the term, and one or two extra lines to make the concept easier to teach. The terms are organized in the same sequence as the expanded guide.

Core memory line: Embedding models search. LLMs generate. RAG connects both. Agents use tools and workflows to complete tasks.

Table of Contents

1. Transformer Model	2. Attention Mechanism
3. Self-Attention	4. Token
5. Tokenization	6. Context Window
7. LLM	8. Generative AI
9. Foundation Model	10. Pre-trained Model
11. Fine-tuning	12. Instruction-Tuned Model
13. Chat Model	14. Base Model
15. Encoder Model	16. Decoder Model
17. Encoder-Decoder Model	18. Embedding
19. Embedding Model	20. Vector
21. Vector Database	22. Semantic Search
23. Keyword Search	24. Hybrid Search
25. RAG	26. Document Loader
27. Chunking	28. Chunk Size
29. Chunk Overlap	30. Retriever
31. Top-K	32. Similarity Score
33. Cosine Similarity	34. Reranker
35. Cross-Encoder	36. Bi-Encoder
37. Multilingual Model	38. Slate Models
39. Granite Models	40. Llama Models
41. Model Size	42. Parameters
43. Training	44. Inference
45. Prompt	46. Prompt Engineering
47. System Prompt	48. Context
49. Grounding	50. Hallucination
51. Temperature	52. Top-p
53. Max Tokens	54. Stop Sequence
55. Autoregressive Model	56. Next Token Prediction
57. Greedy Decoding	58. Sampling
59. Few-shot Prompting	60. Zero-shot Prompting
61. Structured Output	62. Tool Calling
63. Agent	64. Agentic AI
65. Memory	66. Model Context Protocol (MCP)
67. LangChain	68. LangGraph
69. Chain	70. Output Parser
71. RAG vs Fine-tuning	72. Dense Retrieval
73. Sparse Retrieval	74. BM25
75. Metadata	76. Metadata Filtering
77. Query Expansion	78. Multi-query Retrieval
79. Contextual Compression	80. Lost in the Middle Problem
81. Prompt Injection	82. Guardrails
83. Evaluation	84. Faithfulness
85. Relevance	86. Recall

- | | |
|------------------------------|----------------------------------|
| 87. Precision | 88. Latency |
| 89. Quantization | 90. GPU |
| 91. Local LLM | 92. Ollama |
| 93. vLLM | 94. Hugging Face |
| 95. Inference Endpoint | 96. Multimodal Model |
| 97. OCR | 98. Knowledge Base |
| 99. Index | 100. Model Serving |
| 101. LLMOps | 102. AgentOps |
| 103. Model Drift | 104. Prompt Versioning |
| 105. Embedding Drift | 106. Re-indexing |
| 107. Data Ingestion | 108. ETL for RAG |
| 109. Data Cleaning | 110. Deduplication |
| 111. Citation | 112. Source Attribution |
| 113. Retrieval Failure | 114. Generation Failure |
| 115. Small Language Model | 116. Distilled Model |
| 117. Mixture of Experts | 118. Alignment |
| 119. RLHF | 120. DPO |
| 121. LoRA | 122. PEFT |
| 123. Classification Model | 124. Named Entity Recognition |
| 125. Sentiment Analysis | 126. Summarization |
| 127. Translation Model | 128. Code Model |
| 129. Text-to-SQL | 130. Knowledge Graph |
| 131. Graph RAG | 132. Long Context Model |
| 133. Model Router | 134. Orchestration |
| 135. Human-in-the-Loop | 136. Confidence Score |
| 137. PII | 138. Redaction |
| 139. Data Governance | 140. Access Control |
| 141. Multi-tenant RAG | 142. Observability |
| 143. Tracing | 144. Cost Per Token |
| 145. Cache | 146. Evaluation Dataset |
| 147. Golden Dataset | 148. Synthetic Data |
| 149. Benchmark | 150. RAG Architecture |
| 151. Agentic AI Architecture | 152. Quick Mapping Table |
| 153. Best Learning Order | 154. One Final Simple Comparison |

1. Transformer Model

One-liner: The base neural architecture behind most modern LLMs, embedding models, and NLP systems.

Example: A transformer understands that 'bank' means finance in one sentence and river side in another.

Tools / Models: GPT, BERT, T5, Llama, Mistral, IBM Granite, Hugging Face Transformers.

Architecture: Core model architecture layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

2. Attention Mechanism

One-liner: A technique that lets the model focus on the most relevant tokens while processing text.

Example: In 'Python is popular because it is easy', attention connects 'it' to 'Python'.

Tools / Models: Built into GPT, BERT, Llama, Granite, T5, and most transformer models.

Architecture: Core model architecture layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

3. Self-Attention

One-liner: A transformer operation where each token compares itself with other tokens in the same input.

Example: In logs, it connects 'timeout' with 'database connection'.

Tools / Models: All transformer-based LLMs and encoder models use self-attention.

Architecture: Core model architecture layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

4. Token

One-liner: A small unit of text processed by a model, such as a word, subword, number, or punctuation mark.

Example: 'Generative AI' may split into 'Gener', 'ative', and 'AI'.

Tools / Models: tiktoken, Hugging Face tokenizers, SentencePiece.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

5. Tokenization

One-liner: The process of splitting raw text into tokens that a model can process.

Example: 'multilingual' may split into 'multi' and 'lingual'.

Tools / Models: tiktoken, AutoTokenizer, SentencePiece, BPE, WordPiece.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

6. Context Window

One-liner: The maximum number of tokens a model can read and generate within a single request.

Example: A 32K context model can hold the question, retrieved chunks, and answer within 32K tokens.

Tools / Models: LangChain, LlamaIndex, vector databases, LLM APIs, monitoring and evaluation tools.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

7. LLM

One-liner: A Large Language Model that understands and generates human-like text or code.

Example: Ask 'Explain RAG' and the LLM writes the explanation.

Tools / Models: GPT, Claude, Gemini, Llama, IBM Granite, Mistral, Cohere.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

8. Generative AI

One-liner: A broad AI category for creating new content such as text, code, images, audio, and video.

Example: Generate an email, Python code, training notes, or an architecture diagram.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

9. Foundation Model

One-liner: A large pre-trained model that can be adapted to many downstream tasks.

Example: Use foundation model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

10. Pre-trained Model

One-liner: A model that has already learned general patterns from large datasets before you use it.

Example: Use pre-trained model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

11. Fine-tuning

One-liner: Additional training of a pre-trained model on custom data to improve task-specific behavior.

Example: Use fine-tuning when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use fine-tuning for behavior, style, and repeated task patterns. Use RAG for facts, documents, and changing knowledge.

12. Instruction-Tuned Model

One-liner: A model trained to follow instructions such as summarize, classify, extract, or answer.

Example: Use instruction-tuned model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

13. Chat Model

One-liner: A model optimized for conversational interaction using roles such as system, user, assistant, and tool.

Example: Use chat model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

14. Base Model

One-liner: A raw pre-trained model before instruction tuning or chat alignment.

Example: Use base model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

15. Encoder Model

One-liner: A transformer model optimized for understanding and representing input text.

Example: Use encoder model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Core model architecture layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

16. Decoder Model

One-liner: A transformer model optimized for generating text token by token.

Example: Use decoder model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Core model architecture layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

17. Encoder-Decoder Model

One-liner: A transformer model that reads input with an encoder and generates output with a decoder.

Example: Use encoder-decoder model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Core model architecture layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

18. Embedding

One-liner: A numerical vector representation of text meaning.

Example: Embedding is used as part of a practical LLM, RAG, or agentic AI solution.

Tools / Models: LangChain, LlamaIndex, vector databases, LLM APIs, monitoring and evaluation tools.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

19. Embedding Model

One-liner: A model that converts text into embeddings for semantic search and similarity.

Example: Use embedding model when selecting or designing the model layer of an AI system.

Tools / Models: IBM Slate, OpenAI embeddings, BGE, E5, MiniLM, Sentence Transformers, Cohere Embed.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

20. Vector

One-liner: A list of numbers that represents meaning in mathematical form.

Example: In a support assistant, vector helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

21. Vector Database

One-liner: A database optimized to store embeddings and search similar vectors.

Example: Store embedded log chunks and search them when a user asks about an outage.

Tools / Models: FAISS, ChromaDB, Pinecone, Weaviate, Milvus, Qdrant, OpenSearch, pgvector.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

22. Semantic Search

One-liner: Search by meaning rather than exact words.

Example: 'server not responding' retrieves 'VM unreachable'.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

23. Keyword Search

One-liner: Search based on exact terms, lexical matching, or keyword relevance.

Example: Searching 'INC123456' finds exact ticket references.

Tools / Models: Elasticsearch, OpenSearch, Lucene, Solr, BM25.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

24. Hybrid Search

One-liner: A retrieval method combining keyword search and semantic vector search.

Example: A query with case ID plus natural language uses exact and semantic search together.

Tools / Models: OpenSearch, Elasticsearch, Azure AI Search, Weaviate, Pinecone.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

25. RAG

One-liner: Retrieval-Augmented Generation, where relevant content is retrieved before the LLM generates an answer.

Example: Search HR policy first, then ask the LLM to answer reimbursement questions.

Tools / Models: LangChain, LlamaIndex, Haystack, FAISS, Chroma, OpenSearch, watsonx.ai.

Architecture: Retrieval and knowledge-grounding layer.

Extra: The embedding model searches, the vector DB stores, the retriever fetches, and the LLM writes. RAG is best when knowledge changes often.

26. Document Loader

One-liner: A component that reads data from files, APIs, databases, web pages, or enterprise systems.

Example: Load PDFs, ServiceNow tickets, Slack threads, or application logs.

Tools / Models: LangChain loaders, LlamaIndex readers, Unstructured, PyPDF, Apache Tika.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

27. Chunking

One-liner: The process of splitting large documents into smaller searchable sections.

Example: A policy document is split into eligibility, limits, approval, and exception chunks.

Tools / Models: LangChain text splitters, LlamaIndex node parsers, Unstructured.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

28. Chunk Size

One-liner: The amount of text included in each chunk.

Example: In a support assistant, chunk size helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

29. Chunk Overlap

One-liner: Repeated text between adjacent chunks to preserve context across boundaries.

Example: In a support assistant, chunk overlap helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

30. Retriever

One-liner: The component that fetches relevant chunks for a user query.

Example: In a support assistant, retriever helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

31. Top-K

One-liner: The number of top search results returned by a retriever.

Example: In a support assistant, top-k helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

32. Similarity Score

One-liner: A score that shows how close a query and document are in meaning.

Example: In a support assistant, similarity score helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

33. Cosine Similarity

One-liner: A common vector comparison method based on angle between vectors.

Example: In a support assistant, cosine similarity helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

34. Reranker

One-liner: A model or step that reorders retrieved chunks using deeper relevance scoring.

Example: Retrieve 20 chunks, rerank them, and send the best 5 to the LLM.

Tools / Models: Cohere Rerank, BGE reranker, Jina reranker, ColBERT.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

35. Cross-Encoder

One-liner: A model that reads the query and document together to produce a relevance score.

Example: In a support assistant, cross-encoder helps find or prepare the right evidence before the LLM answers.

Tools / Models: sentence-transformers CrossEncoder, BGE reranker, Cohere Rerank.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

36. Bi-Encoder

One-liner: A model that encodes the query and document separately and compares their embeddings.

Example: In a support assistant, bi-encoder helps find or prepare the right evidence before the LLM answers.

Tools / Models: Sentence-BERT, E5, BGE, MiniLM, OpenAI embeddings.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

37. Multilingual Model

One-liner: A model that understands or generates multiple languages.

Example: Use multilingual model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

38. Slate Models

One-liner: IBM-associated models often used for embeddings and enterprise text representation.

Example: Use slate models when selecting or designing the model layer of an AI system.

Tools / Models: IBM watsonx.ai and IBM Slate embedding models.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

39. Granite Models

One-liner: IBM's foundation model family for enterprise text, code, summarization, and RAG workloads.

Example: Use granite models when selecting or designing the model layer of an AI system.

Tools / Models: IBM watsonx.ai, Granite Instruct, Granite Code.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

40. Llama Models

One-liner: Meta's open-weight model family widely used for chat, RAG, fine-tuning, and local AI.

Example: Use llama models when selecting or designing the model layer of an AI system.

Tools / Models: Ollama, Hugging Face, vLLM, llama.cpp, LM Studio.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

41. Model Size

One-liner: The scale of a model, usually represented by parameter count such as 8B or 70B.

Example: Use model size when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

42. Parameters

One-liner: The internal learned weights that store model patterns.

Example: Use parameters when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

43. Training

One-liner: The process of teaching a model by adjusting parameters using data.

Example: Use training when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

44. Inference

One-liner: Using a trained model to produce predictions, summaries, classifications, or answers.

Example: Use inference when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

45. Prompt

One-liner: The instruction or input given to a model.

Example: Use prompt to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

46. Prompt Engineering

One-liner: The practice of designing prompts to improve output quality, format, and reliability.

Example: Use prompt engineering to make model output more reliable, structured, and task-focused.

Tools / Models: LangSmith, PromptLayer, Humanloop, watsonx Prompt Lab.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

47. System Prompt

One-liner: A high-priority instruction that defines assistant behavior and constraints.

Example: Use system prompt to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

48. Context

One-liner: The information available to a model while generating an answer.

Example: Use context to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

49. Grounding

One-liner: Making the model answer from trusted source information instead of guessing.

Example: Use grounding to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

50. Hallucination

One-liner: A confident but false or unsupported model answer.

Example: Use hallucination to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Hallucination is reduced by grounding, citations, low temperature, and evaluation. It cannot be eliminated only by using a bigger model.

51. Temperature

One-liner: A generation setting that controls randomness and creativity.

Example: Use temperature to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

52. Top-p

One-liner: A sampling setting that limits token choices to a probability mass.

Example: Use top-p to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

53. Max Tokens

One-liner: The maximum number of output tokens a model is allowed to generate.

Example: Use max tokens to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

54. Stop Sequence

One-liner: A text pattern that tells the model when to stop generating.

Example: Use stop sequence to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

55. Autoregressive Model

One-liner: A model that generates each token based on previous tokens.

Example: Use autoregressive model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Core model architecture layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

56. Next Token Prediction

One-liner: The training objective where the model learns to predict the next token.

Example: Use next token prediction when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Core model architecture layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

57. Greedy Decoding

One-liner: A decoding method that always selects the most likely next token.

Example: Use greedy decoding to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

58. Sampling

One-liner: A decoding method that chooses tokens probabilistically.

Example: Use sampling to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

59. Few-shot Prompting

One-liner: A prompting method that includes several examples before the actual task.

Example: Use few-shot prompting to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

60. Zero-shot Prompting

One-liner: A prompting method that gives an instruction without examples.

Example: Use zero-shot prompting to make model output more reliable, structured, and task-focused.

Tools / Models: OpenAI, watsonx.ai, Anthropic, Gemini, LangChain prompts, output parsers.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

61. Structured Output

One-liner: A model response constrained to a fixed format such as JSON.

Example: Use structured output to make model output more reliable, structured, and task-focused.

Tools / Models: JSON schema, Pydantic, LangChain parsers, Guardrails AI, Instructor.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

62. Tool Calling

One-liner: A capability where an LLM invokes external functions, APIs, databases, or tools.

Example: The LLM calls `get_ticket_status('INC12345')` before answering.

Tools / Models: OpenAI function calling, LangChain tools, LangGraph ToolNode, MCP.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

63. Agent

One-liner: An LLM-based system that can plan, use tools, observe results, and complete multi-step tasks.

Example: An incident agent searches logs, checks tickets, compares timestamps, and writes RCA.

Tools / Models: LangGraph, LangChain Agents, CrewAI, AutoGen, Semantic Kernel, MCP.

Architecture: Agent workflow, tool-use, and orchestration layer.

Extra: Agents need permissions, logging, and human approval for risky actions. A reliable agent is usually built as a workflow, not a fully uncontrolled loop.

64. Agentic AI

One-liner: AI systems that use planning, tools, memory, and workflows to pursue goals.

Example: A Log Analyzer agent parses logs, finds errors, asks tools, and recommends next steps.

Tools / Models: LangGraph, CrewAI, AutoGen, MCP, Semantic Kernel, watsonx Orchestrate.

Architecture: Agent workflow, tool-use, and orchestration layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

65. Memory

One-liner: Stored information that helps an AI system remember prior context or preferences.

Example: In a Log Analyzer assistant, memory helps coordinate actions beyond a single chatbot response.

Tools / Models: LangChain memory, LangGraph state, Redis, PostgreSQL, vector DBs, Zep.

Architecture: Agent workflow, tool-use, and orchestration layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

66. Model Context Protocol (MCP)

One-liner: A protocol for connecting AI clients with external tools and data sources.

Example: An AI client connects to filesystem, GitHub, or SQLite through MCP servers.

Tools / Models: FastMCP, MCP filesystem, MCP SQLite, MCP GitHub, Claude Desktop MCP.

Architecture: Agent workflow, tool-use, and orchestration layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

67. LangChain

One-liner: A framework for building LLM apps using prompts, chains, tools, retrievers, and agents.

Example: Build a RAG chain with loaders, splitters, retrievers, prompts, and LLM calls.

Tools / Models: LangChain, LCEL, LangSmith, LangChain integrations.

Architecture: Agent workflow, tool-use, and orchestration layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

68. LangGraph

One-liner: A framework for building stateful graph-based LLM workflows and agents.

Example: Build a workflow: retrieve -> decide -> call tool -> summarize.

Tools / Models: LangGraph, LangSmith, LangChain tools.

Architecture: Agent workflow, tool-use, and orchestration layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

69. Chain

One-liner: A fixed sequence of steps in an LLM application.

Example: In a Log Analyzer assistant, chain helps coordinate actions beyond a single chatbot response.

Tools / Models: LangGraph, LangChain, CrewAI, AutoGen, MCP, Semantic Kernel.

Architecture: Agent workflow, tool-use, and orchestration layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

70. Output Parser

One-liner: A component that converts raw model output into structured application-ready data.

Example: Use output parser to make model output more reliable, structured, and task-focused.

Tools / Models: LangChain parsers, Pydantic, Instructor, Guardrails AI.

Architecture: Prompting, generation, and output-control layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

71. RAG vs Fine-tuning

One-liner: A comparison between retrieving knowledge at runtime and training model behavior into weights.

Example: In a support assistant, rag vs fine-tuning helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

72. Dense Retrieval

One-liner: Semantic retrieval using dense embedding vectors.

Example: In a support assistant, dense retrieval helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

73. Sparse Retrieval

One-liner: Keyword-style retrieval using sparse term-based representations.

Example: In a support assistant, sparse retrieval helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

74. BM25

One-liner: A widely used keyword ranking algorithm in search engines.

Example: In a support assistant, bm25 helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

75. Metadata

One-liner: Additional information about a document or chunk, such as source, date, page, or tenant.

Example: In a support assistant, metadata helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

76. Metadata Filtering

One-liner: Restricting retrieval using metadata conditions.

Example: In a support assistant, metadata filtering helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

77. Query Expansion

One-liner: Adding related terms or synonyms to improve search recall.

Example: In a support assistant, query expansion helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

78. Multi-query Retrieval

One-liner: Generating multiple search queries from one user question.

Example: In a support assistant, multi-query retrieval helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

79. Contextual Compression

One-liner: Reducing retrieved content to only the most relevant parts before sending it to the LLM.

Example: In a support assistant, contextual compression helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

80. Lost in the Middle Problem

One-liner: A failure mode where the model misses information placed in the middle of long context.

Example: In a support assistant, lost in the middle problem helps find or prepare the right evidence before the LLM answers.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

81. Prompt Injection

One-liner: A malicious or accidental instruction inside user input or retrieved documents that tries to override rules.

Example: Enterprise RAG uses prompt injection to prevent unsafe, unsupported, or unauthorized responses.

Tools / Models: guardrails, IAM, metadata filters, validators, audit logs, human review.

Architecture: Security, governance, trust, and safety layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

82. Guardrails

One-liner: Rules and checks that keep AI behavior safe, reliable, and compliant.

Example: Enterprise RAG uses guardrails to prevent unsafe, unsupported, or unauthorized responses.

Tools / Models: Guardrails AI, NeMo Guardrails, Pydantic validation, custom policy engines.

Architecture: Security, governance, trust, and safety layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

83. Evaluation

One-liner: Testing the quality of model, retrieval, and final answers.

Example: Production teams monitor evaluation to improve quality, cost, speed, or reliability.

Tools / Models: RAGAS, DeepEval, TruLens, LangSmith, Phoenix, MLflow evaluation.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

84. Faithfulness

One-liner: Whether the answer is supported by the provided context.

Example: Production teams monitor faithfulness to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

85. Relevance

One-liner: How closely retrieved content or generated output matches the user question.

Example: Production teams monitor relevance to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

86. Recall

One-liner: How much of the relevant information the retriever successfully finds.

Example: Production teams monitor recall to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

87. Precision

One-liner: How much of the retrieved information is actually relevant.

Example: Production teams monitor precision to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

88. Latency

One-liner: The time taken by the system to respond.

Example: Production teams monitor latency to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

89. Quantization

One-liner: Reducing model numeric precision to lower memory and compute requirements.

Example: Production teams monitor quantization to improve quality, cost, speed, or reliability.

Tools / Models: llama.cpp, GGUF, GPTQ, AWQ, bitsandbytes, Ollama.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

90. GPU

One-liner: Specialized hardware used to accelerate model training and inference.

Example: Production teams monitor gpu to improve quality, cost, speed, or reliability.

Tools / Models: NVIDIA GPUs, CUDA, AWS/Azure/GCP/IBM Cloud GPU instances.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

91. Local LLM

One-liner: An LLM running on your own machine or server instead of a cloud API.

Example: Production teams monitor local llm to improve quality, cost, speed, or reliability.

Tools / Models: Ollama, LM Studio, llama.cpp, GPT4All, vLLM.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

92. Ollama

One-liner: A tool for running local LLMs easily from the command line or local API.

Example: Run 'ollama run llama3' for a local model demo.

Tools / Models: Ollama CLI and Ollama local API.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

93. vLLM

One-liner: A high-performance serving engine for self-hosted LLM inference.

Example: Serve Llama on GPUs for many concurrent users.

Tools / Models: vLLM, Docker, Kubernetes, NVIDIA GPUs.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

94. Hugging Face

One-liner: A platform and ecosystem for models, datasets, tokenizers, and AI libraries.

Example: Download BGE embeddings or a Llama checkpoint from the Hub.

Tools / Models: Transformers, Datasets, Tokenizers, PEFT, Accelerate, Hub.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

95. Inference Endpoint

One-liner: An API endpoint where a model is deployed for prediction or generation.

Example: Production teams monitor inference endpoint to improve quality, cost, speed, or reliability.

Tools / Models: OpenAI API, IBM watsonx.ai, AWS Bedrock, Azure OpenAI, Vertex AI.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

96. Multimodal Model

One-liner: A model that can process more than one data type, such as text and images.

Example: Use multimodal model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

97. OCR

One-liner: Optical Character Recognition, used to extract text from images and scanned PDFs.

Example: OCR is used as part of a practical LLM, RAG, or agentic AI solution.

Tools / Models: Tesseract, Azure Document Intelligence, AWS Textract, Google Document AI, PaddleOCR.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

98. Knowledge Base

One-liner: The collection of documents, tickets, logs, manuals, or FAQs used by a RAG system.

Example: Production teams monitor knowledge base to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

99. Index

One-liner: A searchable structure built over documents, keywords, or vectors.

Example: Production teams monitor index to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

100. Model Serving

One-liner: Deploying a model so applications can call it through an API or service.

Example: Production teams monitor model serving to improve quality, cost, speed, or reliability.

Tools / Models: vLLM, TGI, Ollama, FastAPI, Triton, KServe, Ray Serve.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

101. LLMOps

One-liner: Operational practices for managing LLM applications in production.

Example: Production teams monitor llmops to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, Phoenix, PromptLayer, TruLens, RAGAS, OpenTelemetry.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

102. AgentOps

One-liner: Monitoring and managing AI agents, their tool calls, loops, and decisions.

Example: Production teams monitor agentops to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, AgentOps.ai, MLflow tracing, OpenTelemetry, LangGraph traces.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

103. Model Drift

One-liner: A decline or change in model performance over time due to changing data or behavior.

Example: Production teams monitor model drift to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

104. Prompt Versioning

One-liner: Tracking prompt changes over time like code versions.

Example: Production teams monitor prompt versioning to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

105. Embedding Drift

One-liner: Retrieval degradation caused by changing embedding models or vector spaces.

Example: Production teams monitor embedding drift to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Query and document vectors should come from the same embedding model. If the embedding model changes, recreate document embeddings.

106. Re-indexing

One-liner: Rebuilding the search index after documents, chunks, metadata, or embeddings change.

Example: Production teams monitor re-indexing to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

107. Data Ingestion

One-liner: Bringing source data into a RAG or AI system.

Example: Production teams monitor data ingestion to improve quality, cost, speed, or reliability.

Tools / Models: LangChain, LlamaIndex, Airflow, Prefect, Unstructured, Pandas.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

108. ETL for RAG

One-liner: Extracting, transforming, and loading data into a RAG-ready index.

Example: Production teams monitor etl for rag to improve quality, cost, speed, or reliability.

Tools / Models: Airflow, Prefect, Dagster, OpenSearch indexing jobs, custom Python.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

109. Data Cleaning

One-liner: Removing noise and formatting problems from raw data before indexing.

Example: Production teams monitor data cleaning to improve quality, cost, speed, or reliability.

Tools / Models: regex, BeautifulSoup, Pandas, Unstructured, Apache Tika.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

110. Deduplication

One-liner: Removing duplicate documents or chunks from a knowledge base.

Example: Production teams monitor deduplication to improve quality, cost, speed, or reliability.

Tools / Models: hashing, MinHash, SimHash, embedding similarity, Pandas.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

111. Citation

One-liner: A reference showing the source behind an answer.

Example: Enterprise RAG uses citation to prevent unsafe, unsupported, or unauthorized responses.

Tools / Models: guardrails, IAM, metadata filters, validators, audit logs, human review.

Architecture: Security, governance, trust, and safety layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

112. Source Attribution

One-liner: Identifying exactly where the answer evidence came from.

Example: Enterprise RAG uses source attribution to prevent unsafe, unsupported, or unauthorized responses.

Tools / Models: guardrails, IAM, metadata filters, validators, audit logs, human review.

Architecture: Security, governance, trust, and safety layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

113. Retrieval Failure

One-liner: When the system fails to fetch the correct evidence.

Example: Enterprise RAG uses retrieval failure to prevent unsafe, unsupported, or unauthorized responses.

Tools / Models: guardrails, IAM, metadata filters, validators, audit logs, human review.

Architecture: Security, governance, trust, and safety layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

114. Generation Failure

One-liner: When the LLM has evidence but produces a wrong, incomplete, or malformed answer.

Example: Enterprise RAG uses generation failure to prevent unsafe, unsupported, or unauthorized responses.

Tools / Models: guardrails, IAM, metadata filters, validators, audit logs, human review.

Architecture: Security, governance, trust, and safety layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

115. Small Language Model

One-liner: A smaller model optimized for speed, cost, or local deployment.

Example: Use small language model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

116. Distilled Model

One-liner: A smaller model trained to imitate a larger model.

Example: Use distilled model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

117. Mixture of Experts

One-liner: An architecture with multiple expert subnetworks where only some activate per input.

Example: Use mixture of experts when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Core model architecture layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

118. Alignment

One-liner: Training or tuning a model to behave according to human expectations and safety rules.

Example: Use alignment when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

119. RLHF

One-liner: Reinforcement Learning from Human Feedback, used to align model behavior.

Example: Use rlhf when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

120. DPO

One-liner: Direct Preference Optimization, a simpler preference-based alignment technique.

Example: Use dpo when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

121. LoRA

One-liner: Low-Rank Adaptation, a lightweight fine-tuning method.

Example: Use lora when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

122. PEFT

One-liner: Parameter-Efficient Fine-Tuning, a family of low-cost tuning techniques.

Example: Use peft when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

123. Classification Model

One-liner: A model that assigns labels to text or records.

Example: Use classification model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

124. Named Entity Recognition

One-liner: Detecting entities such as names, dates, servers, tickets, and locations in text.

Example: Use named entity recognition when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

125. Sentiment Analysis

One-liner: Detecting tone or emotion such as positive, negative, neutral, urgent, or frustrated.

Example: Use sentiment analysis when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

126. Summarization

One-liner: Converting long content into a shorter version.

Example: Use summarization when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

127. Translation Model

One-liner: A model that converts text from one language to another.

Example: Use translation model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

128. Code Model

One-liner: A model optimized for programming tasks.

Example: Use code model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Model selection, customization, or inference layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

129. Text-to-SQL

One-liner: Converting natural language questions into SQL queries.

Example: 'Show top 5 customers by revenue' becomes a SELECT query.

Tools / Models: LangChain SQLDatabaseChain, LlamaIndex SQL query engine, SQL validators.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

130. Knowledge Graph

One-liner: A graph of entities and relationships.

Example: Customer -> owns -> VM -> hosted_on -> Host -> located_in -> Data Center.

Tools / Models: Neo4j, Amazon Neptune, ArangoDB, TigerGraph, RDF stores.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

131. Graph RAG

One-liner: RAG enhanced with graph-based entity and relationship retrieval.

Example: Find impacted customers by traversing Migration -> Host -> VM -> Customer.

Tools / Models: Neo4j, LlamaIndex Graph RAG, LangChain graph integrations.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

132. Long Context Model

One-liner: A model that can process very large amounts of text in one request.

Example: Use long context model when selecting or designing the model layer of an AI system.

Tools / Models: Hugging Face, PyTorch, Transformers, watsonx.ai, OpenAI API, local LLM runtimes.

Architecture: Core model architecture layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

133. Model Router

One-liner: A component that chooses the best model for each task.

Example: In a Log Analyzer assistant, model router helps coordinate actions beyond a single chatbot response.

Tools / Models: LangGraph, LangChain, CrewAI, AutoGen, MCP, Semantic Kernel.

Architecture: Agent workflow, tool-use, and orchestration layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

134. Orchestration

One-liner: Coordinating multiple AI components, tools, models, and workflows.

Example: In a Log Analyzer assistant, orchestration helps coordinate actions beyond a single chatbot response.

Tools / Models: LangGraph, LangChain, CrewAI, AutoGen, MCP, Semantic Kernel.

Architecture: Agent workflow, tool-use, and orchestration layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

135. Human-in-the-Loop

One-liner: A design where humans review or approve AI outputs or actions.

Example: In a Log Analyzer assistant, human-in-the-loop helps coordinate actions beyond a single chatbot response.

Tools / Models: LangGraph, LangChain, CrewAI, AutoGen, MCP, Semantic Kernel.

Architecture: Agent workflow, tool-use, and orchestration layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

136. Confidence Score

One-liner: A numeric estimate of how certain a system is about an answer or classification.

Example: Confidence Score is used as part of a practical LLM, RAG, or agentic AI solution.

Tools / Models: LangChain, LlamaIndex, vector databases, LLM APIs, monitoring and evaluation tools.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

137. PII

One-liner: Personally Identifiable Information such as name, email, phone, address, or ID.

Example: Enterprise RAG uses pii to prevent unsafe, unsupported, or unauthorized responses.

Tools / Models: guardrails, IAM, metadata filters, validators, audit logs, human review.

Architecture: Security, governance, trust, and safety layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

138. Redaction

One-liner: Masking or removing sensitive information before storage or model use.

Example: Enterprise RAG uses redaction to prevent unsafe, unsupported, or unauthorized responses.

Tools / Models: guardrails, IAM, metadata filters, validators, audit logs, human review.

Architecture: Security, governance, trust, and safety layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

139. Data Governance

One-liner: Policies and controls for responsible data usage, access, retention, and auditability.

Example: Enterprise RAG uses data governance to prevent unsafe, unsupported, or unauthorized responses.

Tools / Models: IAM, RBAC, ABAC, audit logs, Collibra, OpenMetadata, Apache Ranger.

Architecture: Security, governance, trust, and safety layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

140. Access Control

One-liner: Rules that determine who can access which documents or outputs.

Example: Enterprise RAG uses access control to prevent unsafe, unsupported, or unauthorized responses.

Tools / Models: guardrails, IAM, metadata filters, validators, audit logs, human review.

Architecture: Security, governance, trust, and safety layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

141. Multi-tenant RAG

One-liner: A RAG system serving multiple clients or groups while isolating their data.

Example: Enterprise RAG uses multi-tenant rag to prevent unsafe, unsupported, or unauthorized responses.

Tools / Models: guardrails, IAM, metadata filters, validators, audit logs, human review.

Architecture: Security, governance, trust, and safety layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

142. Observability

One-liner: Monitoring prompts, responses, retrieval, tool calls, errors, cost, and latency.

Example: Production teams monitor observability to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, OpenTelemetry, MLflow tracing, Phoenix, Datadog, Grafana.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

143. Tracing

One-liner: Recording each step of an LLM or agent workflow.

Example: Production teams monitor tracing to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow tracing, OpenTelemetry, Phoenix, TruLens, AgentOps.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

144. Cost Per Token

One-liner: The price charged based on model input and output token usage.

Example: Production teams monitor cost per token to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

145. Cache

One-liner: Stored results used to reduce repeated computation, latency, and cost.

Example: Production teams monitor cache to improve quality, cost, speed, or reliability.

Tools / Models: Redis, SQLite, PostgreSQL, LangChain cache, GPTCache, semantic cache.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

146. Evaluation Dataset

One-liner: A set of test questions and expected answers used to evaluate quality.

Example: Production teams monitor evaluation dataset to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

147. Golden Dataset

One-liner: A high-quality expert-verified evaluation dataset.

Example: Production teams monitor golden dataset to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

148. Synthetic Data

One-liner: Artificially generated data used for testing, demos, or training.

Example: Production teams monitor synthetic data to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

149. Benchmark

One-liner: A standard test used to compare model capabilities.

Example: Production teams monitor benchmark to improve quality, cost, speed, or reliability.

Tools / Models: LangSmith, MLflow, OpenTelemetry, Phoenix, RAGAS, monitoring dashboards.

Architecture: Production operations, monitoring, optimization, or evaluation layer.

Extra: Use this concept with examples from documents, tickets, logs, or internal knowledge bases. In production, measure quality with retrieval and answer evaluation.

150. RAG Architecture

One-liner: A reference design showing how documents, embeddings, retrieval, and LLM generation connect.

Example: Documents -> chunks -> embeddings -> vector DB -> retriever -> LLM -> answer.

Tools / Models: LangChain, LlamaIndex, FAISS, Chroma, OpenSearch, Elasticsearch, vector DBs.

Architecture: Retrieval and knowledge-grounding layer.

Extra: This is a teaching summary rather than a single tool. Use it as a slide, revision page, or whiteboard explanation.

151. Agentic AI Architecture

One-liner: A reference design showing how an agent plans, calls tools, observes results, and completes goals.

Example: Goal -> planner -> tools -> observations -> final report.

Tools / Models: LangGraph, LangChain, CrewAI, AutoGen, MCP, Semantic Kernel.

Architecture: Agent workflow, tool-use, and orchestration layer.

Extra: This is a teaching summary rather than a single tool. Use it as a slide, revision page, or whiteboard explanation.

152. Quick Mapping Table

One-liner: A quick mapping of common terms to their practical roles.

Example: LLM writes, embedding model searches, vector DB stores, retriever fetches.

Tools / Models: LangChain, LlamaIndex, vector databases, LLM APIs, monitoring and evaluation tools.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: This is a teaching summary rather than a single tool. Use it as a slide, revision page, or whiteboard explanation.

153. Best Learning Order

One-liner: A recommended sequence for learning LLM, RAG, and agentic AI concepts.

Example: Learn tokens first, then transformers, embeddings, RAG, agents, and LLMOps.

Tools / Models: LangChain, LlamaIndex, vector databases, LLM APIs, monitoring and evaluation tools.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: This is a teaching summary rather than a single tool. Use it as a slide, revision page, or whiteboard explanation.

154. One Final Simple Comparison

One-liner: A compact summary comparing LLMs, embedding models, vector DBs, RAG, and agents.

Example: RAG is an open-book exam: the embedding model finds pages and the LLM writes the answer.

Tools / Models: LangChain, LlamaIndex, vector databases, LLM APIs, monitoring and evaluation tools.

Architecture: Supporting concept used across LLM, RAG, and agent systems.

Extra: This is a teaching summary rather than a single tool. Use it as a slide, revision page, or whiteboard explanation.